

Title:

SQL & LDAP Injection

Course:

Software Security

Institution: ECE Paris

Instructor:

Mr. Yaya Doumbia

Academic Year: 2025-2026

By

OFUNNEKA JENNIFER OKONKWOABUTU

DATE: 6th February, 2026

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
1.0 INTRODUCTION.....	3
1.1 Purpose of this Project.....	3
1.2 Objectives of the project.....	3
1.3 Source Code Reference.....	3
2.0 DEVELOPMENT ENVIRONMENT CONFIGURATION.....	4
2.1 Tools and Technologies Used.....	4
2.2 Environment setup.....	5
Step 1: Install Required Software.....	5
Step 2: Use WAMP to create database, user accounts, database login user.....	6
Step 3: Create a test folder to host our application.....	8
Step 4: Testing the application.....	10
2.3 Conclusion:.....	11
3.0 EXPLOITING VULNERABILITIES - SQL INJECTION.....	12
3.1 Application Scan Using Owasp Zap.....	12
3.2 Sql Injection Attempt Before Vulnerability Fix.....	15
3.3 Sql Injection Test After Patching/Code Fix.....	18
3.4 Conclusion:.....	21
4.0 LDAP INJECTIONS.....	22
4.1 Environment Configuration.....	22
Step 1: Install Required Software.....	22
Step 2: Verify Apache supports LDAP.....	22
Step 3: Create PHP Application in XAMPP Directory.....	23
Step 4: Setup OpenLDAP Server with Directories.....	26
Step 5: Confirm Connection between XAMPP Application & LDAP Directory.....	30
4.2 LDAP Authentication.....	31
4.3 Perform LDAP Injection Bypass.....	32
4.4 Vulnerability Fix.....	35
4.5 Verification and Re-Testing.....	36
4.6 Conclusion:.....	38
FINAL CONCLUSION.....	39
APPENDIX.....	40
Appendix A – SQL Injection Payloads Used.....	40
Appendix B – LDAP Injection Payloads Used.....	40
Appendix C – Secure Coding Techniques Implemented.....	40
Appendix D – Table of Findings.....	40

1.0 INTRODUCTION

1.1 Purpose of this Project

The purpose of this lab is to practically demonstrate injection vulnerabilities, specifically SQL injection and LDAP injection as defined by OWASP Top 10 Web Application Security Risks.

The project focuses on deploying vulnerable web applications, exploiting the vulnerabilities using industry-standards tools, and implementing secure fixes to mitigate the identified risks.

This operational document describes what was implemented, how it was tested, the vulnerabilities discovered, and the remediation measures applied.

1.2 Objectives of the project

Functional Objectives:

1. To understand the fundamentals of SQL and LDAP injection attacks.
2. To deploy intentionally vulnerable PHP_based authentication systems
3. Exploit vulnerabilities using OWASP ZAP
4. Demonstrate authentication bypass attacks.
5. Apply secure coding techniques to prevent injections.
6. Validate fixes through retesting and vulnerability scans

1.3 Source Code Reference

The complete source code for this project is available in a public Github repository and can be used to reproduce the system described in this report.

Repository Link:

[JanieAbutu/Software_Application-Security-Projects](https://github.com/JanieAbutu/Software_Application-Security-Projects)

2.0 DEVELOPMENT ENVIRONMENT CONFIGURATION

2.1 Tools and Technologies Used

Operating Systems

- **Windows:**
Primary development machine
- **Ubuntu:**
Used to configure and run the OpenLDAP server

Web Servers

- **WAMP (Apache, MySQL, PHP):**
Provides Apache, PHP, and MySQL environments required to host and run the web application used for SQL injection.
- **XAMPP (Apache, MySQL, PHP, Perl):**
Provides Apache and PHP environments required to host and run the web application used for LDAP injection.

Database and Directories

- **MySQL:**
Database used for SQL injection testing
- **OpenLDAP:**
Directory service used for LDAP injection testing

Security Testing Tool

- **OWASP Zed Attack Proxy (ZAP):**
Used to intercept and analyze injection vulnerabilities.

Programming Language

- **PHP:**
Used to build the test web application

Summary:

The development environment consists of a Windows-based WAMP server hosting a deliberately vulnerable PHP application connected to a MySQL database. An Ubuntu virtual machine hosts OpenLDAP for LDAP injection testing. OWASP ZAP is used to perform vulnerability scanning against the web application. This configuration provides a controlled environment for demonstrating SQL injection and LDAP injection attacks and their impact.

2.2 Environment setup

This section describes how the development environment was configured before starting the project. A proper setup ensures the web application runs correctly, the database and authentication work as expected, security tools can interact with the system, and injection vulnerabilities can be effectively demonstrated.

Step 1: Install Required Software

A. Install WAMP Server

Actions Performed:

- Download WAMP from <https://www.wampserver.com/>
- Execute the installer, select the components and install
- Verify the installation by starting WAMP, confirm the WAMP icon on windows icon tray turns green

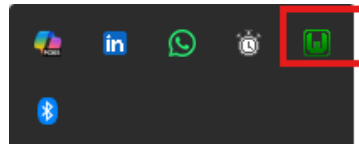


Fig 2.2.1 WAMP server successfully installed and running

B. Install OWASP ZAP

Actions Performed:

- Download OWASP ZAP from <https://www.zaproxy.org/download/>
- Execute the installer, select the components and install

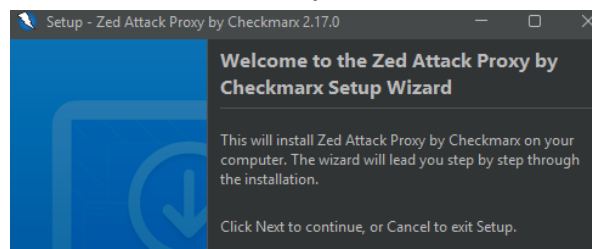


Fig 2.2.2 OWASP ZAP installer

- Launch

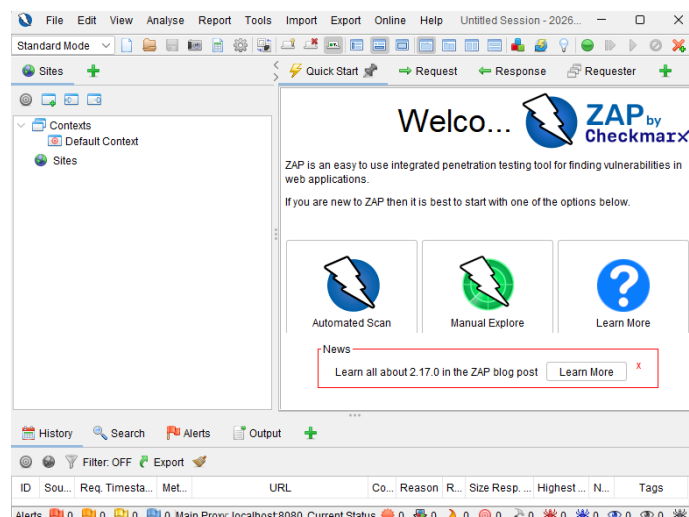


Fig 2.2.3 OWASP ZAP successfully installed and running

Step 2: Use WAMP to create database, user accounts, database login user

Purpose:

The purpose of this step is to create a test database, a user table, test user accounts, and a dedicated database login user. This database will be used by the vulnerable PHP application for authentication and testing.

- Open MySQL console by left-clicking WAMP icon and selecting MySQL
- Use root and enter

Actions Performed:

1. Create database named **zap_test**,
run **create database zap_test**;

```
mysql> create database zap_test;  
Query OK, 1 row affected (0.01 sec)
```

Fig 2.2.2 Command run to create the database 'zap_test'

OUTPUT

Database zap_test created successfully.

```
mysql> SHOW DATABASES;  
+-----+  
| Database |  
+-----+  
| information_schema |  
| mysql |  
| performance_schema |  
| sys |  
| uzuri |  
| wealthdb |  
| zap_test |  
+-----+  
7 rows in set (0.06 sec)
```

Fig 2.2.3 Command run to verify that the database 'zap_test' was created

2. Create table 'zap_users' and add user accounts on the database,
 - run **use zap_test** to select the database

```
mysql> use zap_test  
Database changed
```

Fig 2.2.4 Database selected

- Run **create table zap_users(id int primary key auto_increment, username varchar (20), password varchar(20));**

```
mysql> create table zap_users(id int primary key auto_increment, username varchar (20), password varchar(20));  
Query OK, 0 rows affected (0.02 sec)
```

Fig 2.2.5 Table creation command run

- Run **insert into zap_users(username, password)values('ismaila', 'passer');** to create users

```
mysql>
mysql> insert into zap_users(username, password)values('ismaila', 'passer');
Query OK, 1 row affected (0.02 sec)

mysql> insert into zap_users(username, password)values('jennifer', 'passer2');
Query OK, 1 row affected (0.00 sec)

mysql> insert into zap_users(username, password)values('umar', 'passer3');
Query OK, 1 row affected (0.00 sec)

mysql> insert into zap_users(username, password)values('maina', 'passer4');
Query OK, 1 row affected (0.00 sec)

mysql> insert into zap_users(username, password)values('ayoub', 'passer5');
Query OK, 1 row affected (0.00 sec)

mysql>
```

Fig 2.2.6: Insert commands run to create 5 different user accounts.

OUTPUT:

5 user accounts successfully inserted

```
mysql> SELECT * FROM zap_users;
+----+-----+-----+
| id | username | password |
+----+-----+-----+
| 1  | ismaila  | passer   |
| 2  | jennifer | passer2  |
| 3  | umar     | passer3  |
| 4  | maina    | passer4  |
| 5  | ayoub    | passer5  |
+----+-----+-----+
5 rows in set (0.00 sec)
```

Fig 2.2.7: Created user accounts verified

3. Create the database login user

- Run **CREATE USER 'janie'@'%' IDENTIFIED BY 'passer';**
- Run **GRANT ALL PRIVILEGES ON zap_test.* TO 'janie'@'%';**
- **FLUSH PRIVILEGES;**

```
mysql>
mysql> CREATE USER 'janie'@'%' IDENTIFIED BY 'passer';
Query OK, 0 rows affected (0.29 sec)

mysql>
mysql> GRANT ALL PRIVILEGES ON zap_test.* TO 'janie'@'%';
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.03 sec)
```

Fig 2.2.8: Database login user created

OUTPUT:

Run below to command to login to database as 'janie'

"C:\wamp64\bin\mysql\mysql9.1.0\bin\mysql.exe" -u janie -p --port=3309

zap_test and to confirm user account logged in, run **SELECT CURRENT_USER();**

```
C:\Windows\System32>"C:\wamp64\bin\mysql\mysql9.1.0\bin\mysql.exe" -u janie -p --port=3309 zap_test
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 17
Server version: 9.1.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
mysql> SELECT CURRENT_USER();
+-----+
| CURRENT_USER() |
+-----+
| janie@%        |
+-----+
1 row in set (0.01 sec)

mysql>
```

Fig 2.2.9: Successful login by database user account 'janie'

Step 3: Create a test folder to host our application

Purpose:

To create a directory inside the WAMP www folder where the vulnerable PHP application will be hosted.

Actions Performed:

A. Create folder named 'test' in C:\wamp64\www\Software Security\Software Security Projects\SQL & LDAP injection\test

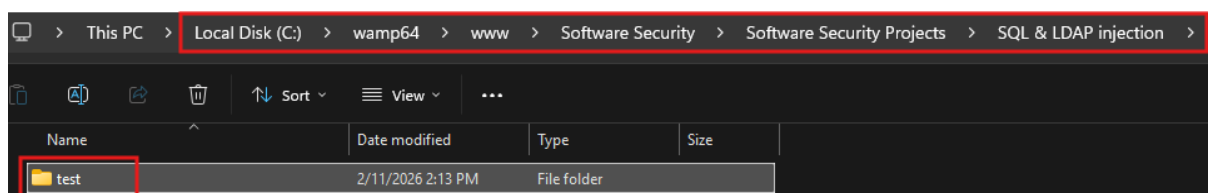
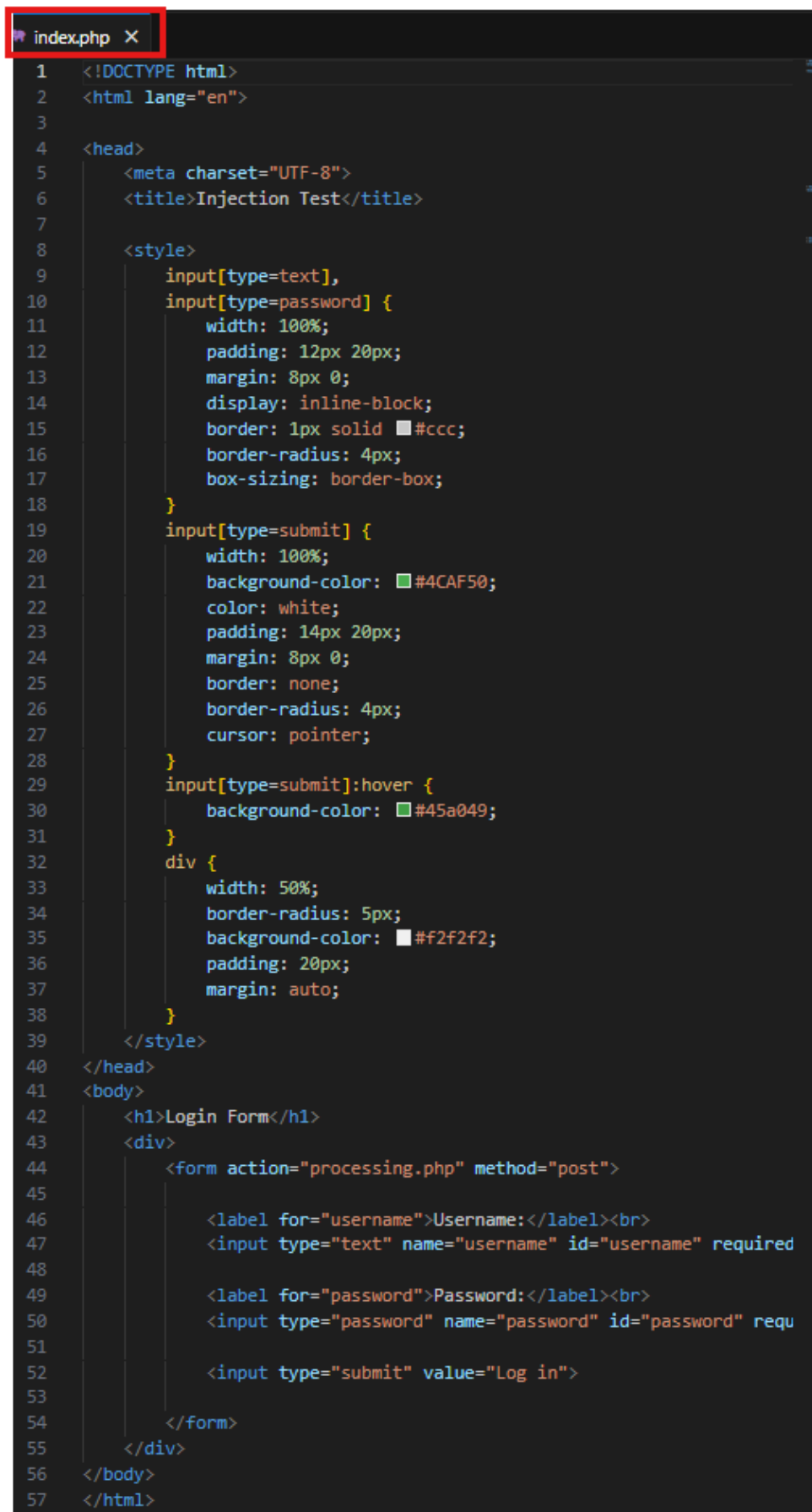


Fig 3.1: Created folder 'test' in the directory 'C:\wamp64\www\Software Security\Software Security Projects\SQL & LDAP injection'

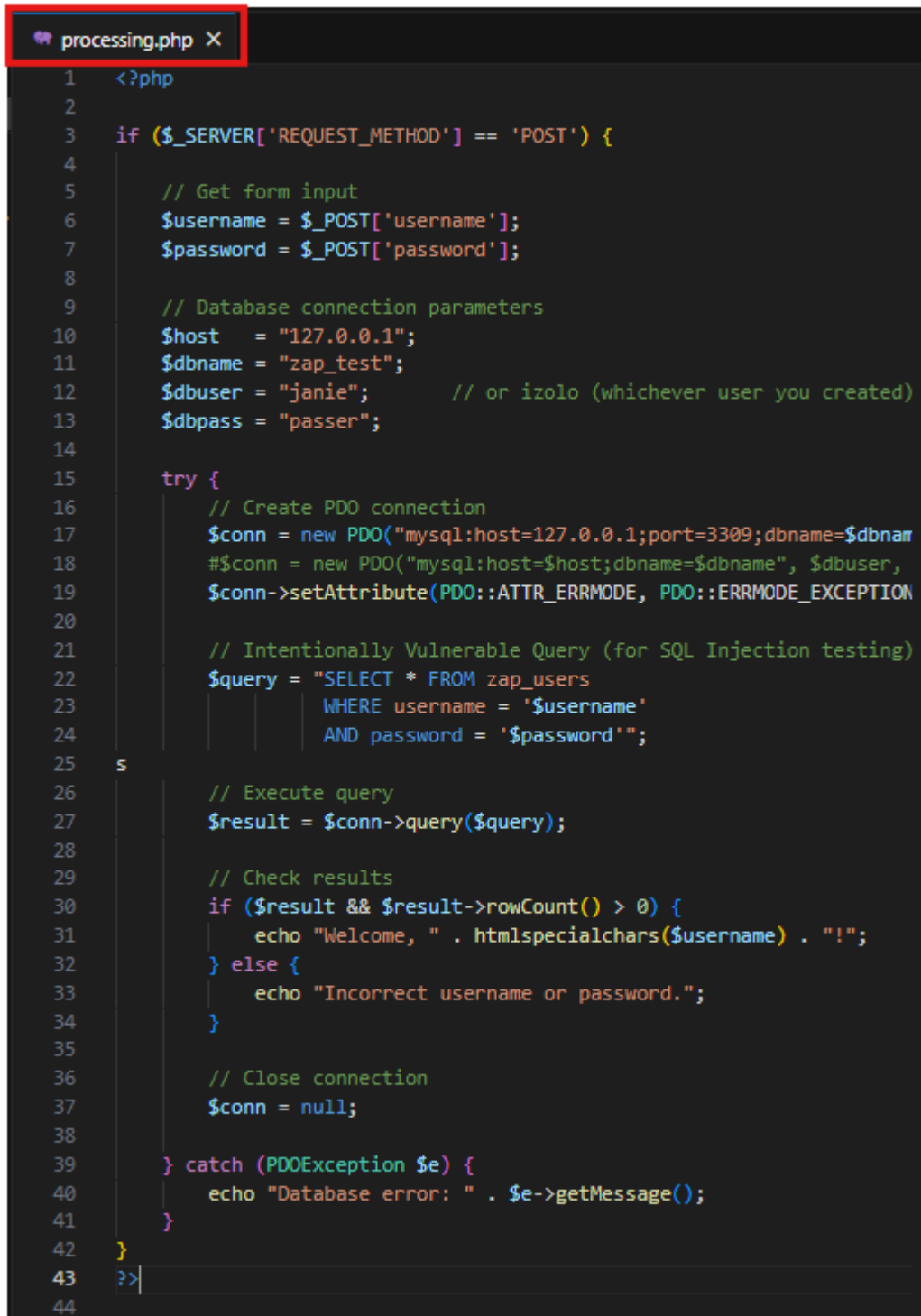
B. Create `index.php` in the test folder

A screenshot of a code editor with a dark background. At the top, a tab labeled 'index.php' with a close button 'X' is highlighted with a red rectangle. The code is a PHP file that outputs an HTML login form. It includes a DOCTYPE declaration, a meta charset, a title 'Injection Test', and a style block with CSS rules for text inputs, password inputs, submit buttons, and a container div. The form itself is a POST request to 'processing.php' with fields for 'username' and 'password', and a 'Log in' submit button.

```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5     <meta charset="UTF-8">
6     <title>Injection Test</title>
7
8     <style>
9         input[type=text],
10        input[type=password] {
11            width: 100%;
12            padding: 12px 20px;
13            margin: 8px 0;
14            display: inline-block;
15            border: 1px solid #ccc;
16            border-radius: 4px;
17            box-sizing: border-box;
18        }
19        input[type=submit] {
20            width: 100%;
21            background-color: #4CAF50;
22            color: white;
23            padding: 14px 20px;
24            margin: 8px 0;
25            border: none;
26            border-radius: 4px;
27            cursor: pointer;
28        }
29        input[type=submit]:hover {
30            background-color: #45a049;
31        }
32        div {
33            width: 50%;
34            border-radius: 5px;
35            background-color: #f2f2f2;
36            padding: 20px;
37            margin: auto;
38        }
39    </style>
40 </head>
41 <body>
42     <h1>Login Form</h1>
43     <div>
44         <form action="processing.php" method="post">
45
46             <label for="username">Username:</label><br>
47             <input type="text" name="username" id="username" required
48
49             <label for="password">Password:</label><br>
50             <input type="password" name="password" id="password" requ
51
52             <input type="submit" value="Log in">
53
54         </form>
55     </div>
56 </body>
57 </html>
```

Fig 3.2: Created file 'index.php' which contains the login form

C. Create processing.php in the test folder



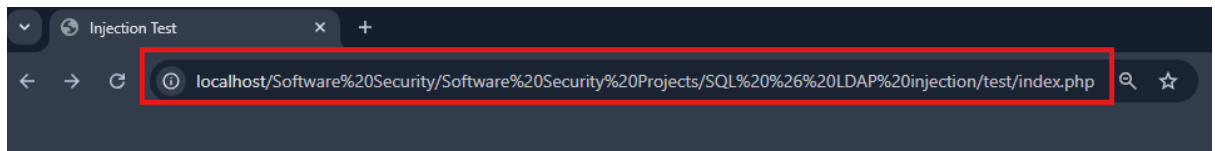
```
1  <?php
2
3  if ($_SERVER['REQUEST_METHOD'] == 'POST') {
4
5      // Get form input
6      $username = $_POST['username'];
7      $password = $_POST['password'];
8
9      // Database connection parameters
10     $host = "127.0.0.1";
11     $dbname = "zap_test";
12     $dbuser = "janie"; // or izolo (whichever user you created)
13     $dbpass = "passer";
14
15     try {
16         // Create PDO connection
17         $conn = new PDO("mysql:host=127.0.0.1;port=3309;dbname=$dbname", $dbuser, $dbpass);
18         # $conn = new PDO("mysql:host=$host;dbname=$dbname", $dbuser, $dbpass);
19         $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
20
21         // Intentionally Vulnerable Query (for SQL Injection testing)
22         $query = "SELECT * FROM zap_users
23                 WHERE username = '$username'
24                 AND password = '$password'";
25
26         // Execute query
27         $result = $conn->query($query);
28
29         // Check results
30         if ($result && $result->rowCount() > 0) {
31             echo "Welcome, " . htmlspecialchars($username) . "!";
32         } else {
33             echo "Incorrect username or password.";
34         }
35
36         // Close connection
37         $conn = null;
38
39     } catch (PDOException $e) {
40         echo "Database error: " . $e->getMessage();
41     }
42 }
43 ?>
```

Fig 3.3: Created file 'processing.php' in the test directory

Step 4: Testing the application

A. Open the url in browser -

<http://localhost/Software%20Security/Software%20Security%20Projects/SQL%20%26%20LDAP%20injection/test/index.php>



Login Form

A screenshot of a login form. It has two input fields: 'Username:' and 'Password:'. Below the fields is a green button labeled 'Log in'.

Fig 4.1: Login form opened on the browser confirming application accessibility

B. Login in with valid credentials stored in database

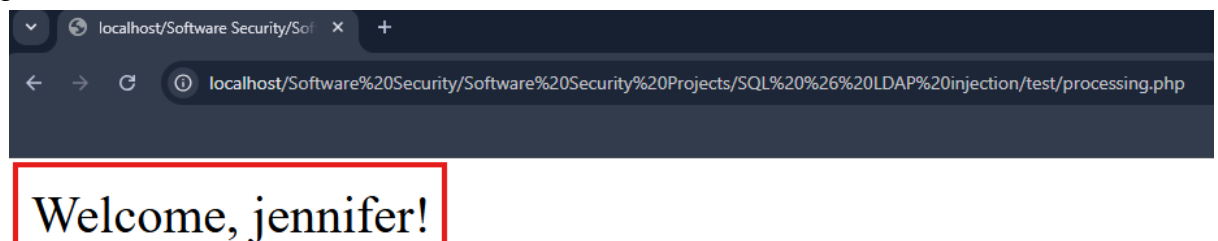


Fig 4.2: Successful Login using one of the user credentials stored in the database

C. Login with invalid credentials

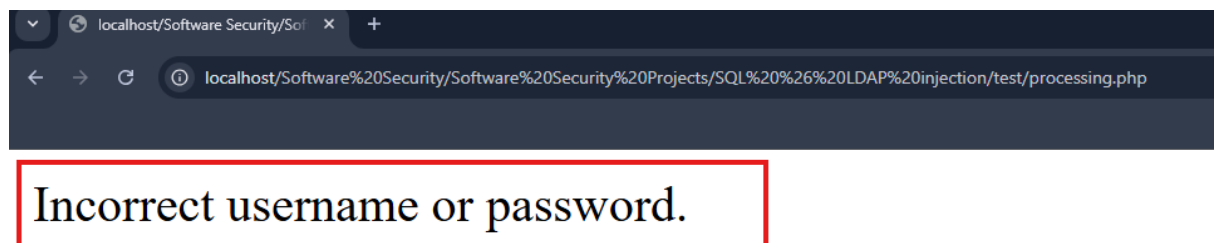


Fig 4.3: Failed Login using wrong username and password, proof that the application is working properly

2.3 Conclusion:

The development environment was successfully configured by installing and verifying WAMP, creating the zap_test database and zap_users table, inserting multiple test user accounts, and configuring a dedicated database login user. The PHP login application was deployed and tested successfully using both valid and invalid credentials. The system is now fully operational and ready for SQL injection testing and vulnerability assessment using OWASP ZAP.

3.0 EXPLOITING VULNERABILITIES - SQL INJECTION

3.1 Application Scan Using Owasp Zap

Purpose:

To use the OWASP ZAP tool to determine whether the PHP login application is vulnerable to SQL injection attacks by intercepting and manipulating HTTP requests.

Actions Performed:

- Click Automated scan
- Enter URL of our login web application
- Click Attack

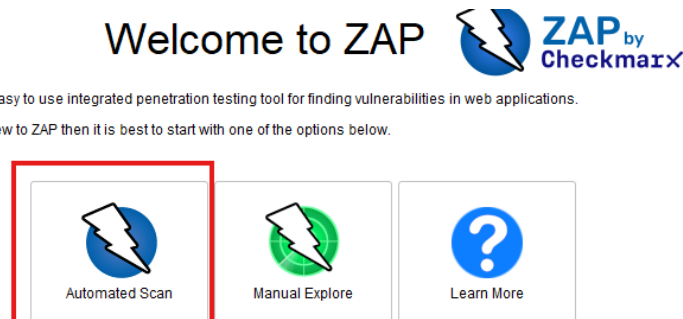


Fig 3.1.1 Automated Scan selected from OWASP ZAP GUI

INPUT:

- Add the URL to scan in the 'URL to attack' input field
- Click Attack

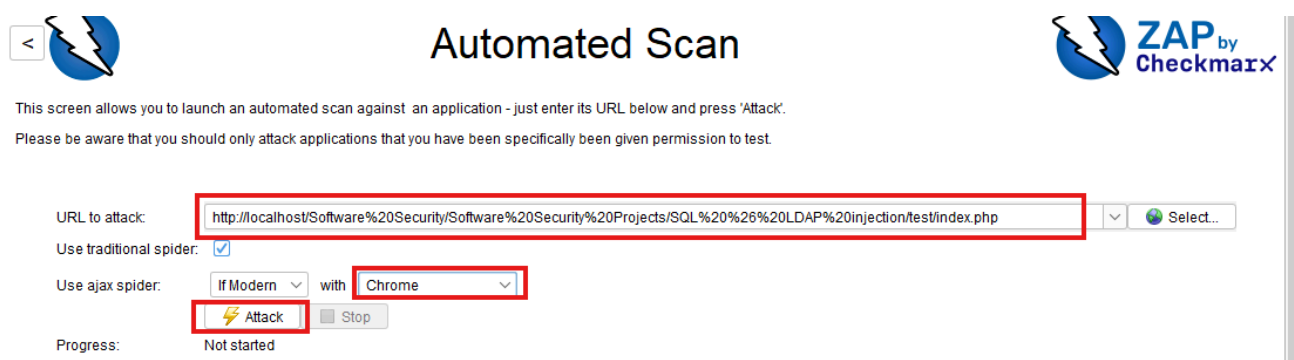


Fig 3.1.2 Required Fields filled and selected on automated scan page

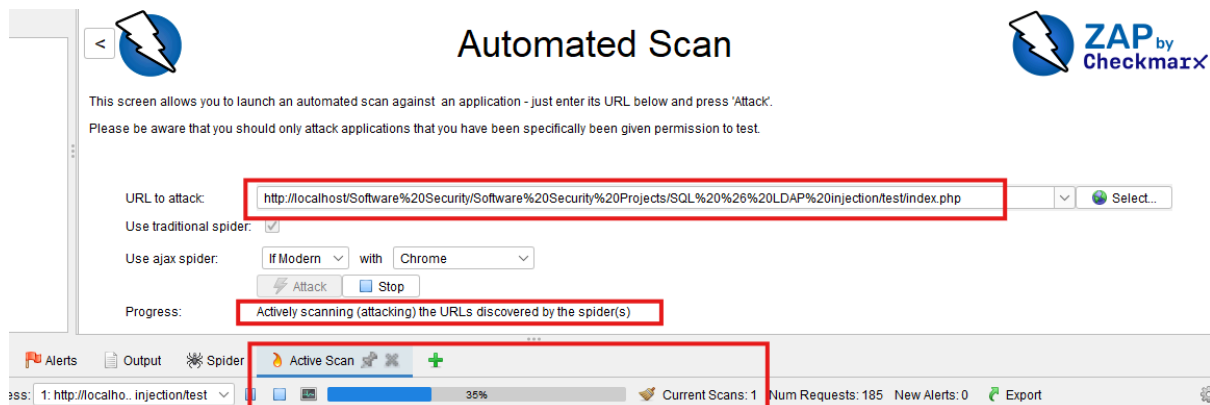


Fig 3.1.3 Automated scan in progress

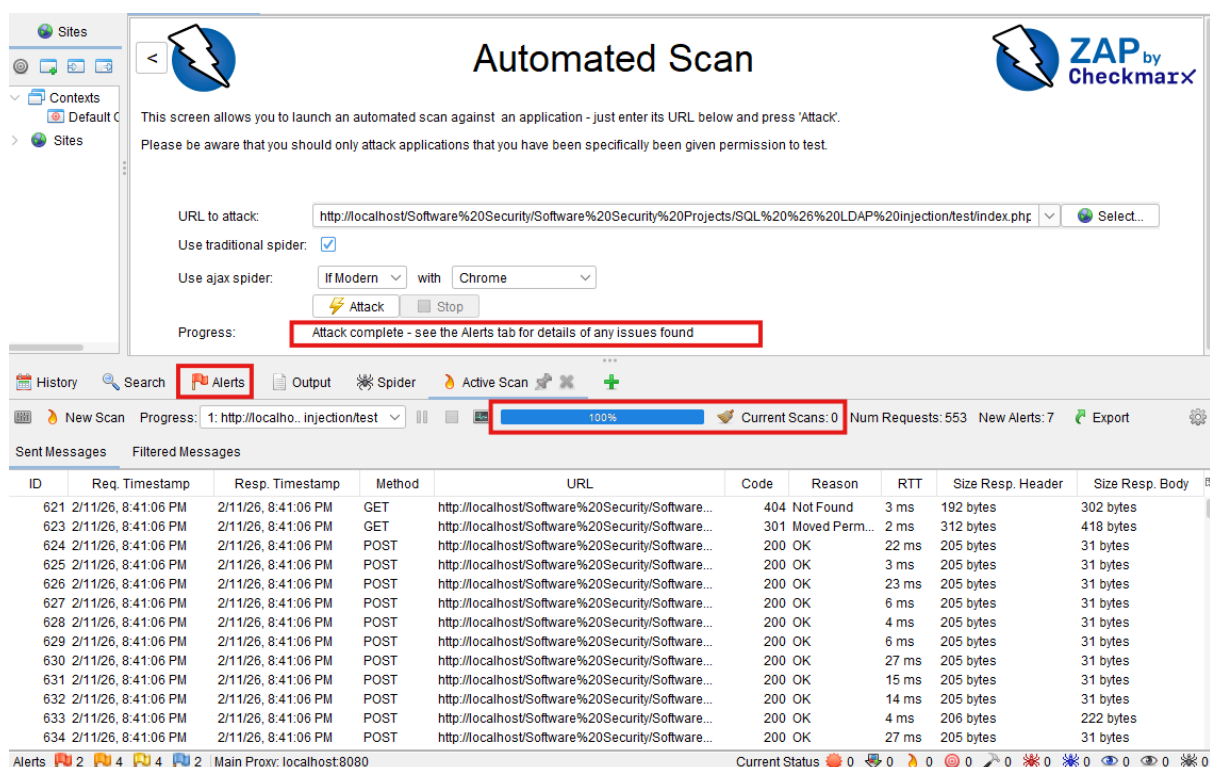


Fig 3.1.4 Scan completed and results displayed

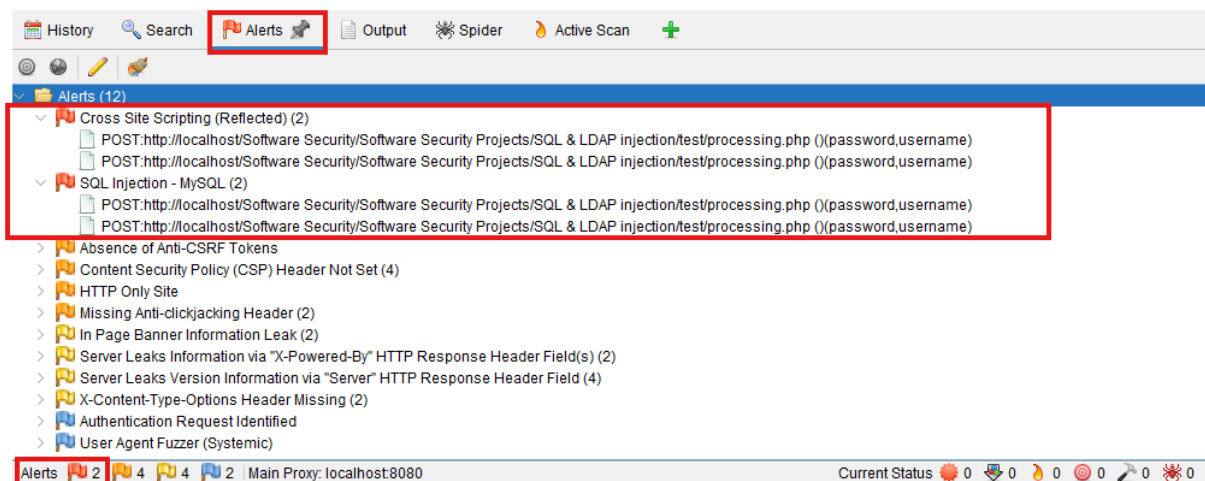


Fig 3.1.5 Alerts showing SQL Injection Vulnerability found

3.2 Sql Injection Attempt Before Vulnerability Fix

Purpose:

The purpose of this test was to verify whether the login functionality of the web application was vulnerable to SQL Injection attacks.

Objective:

- Determine if authentication bypass was possible.
- Determine database manipulation can be achieved through injection

Actions Performed:

A. Attempt Login using Valid Username and Password ZAP' OR '1'='1' --

Purpose: To test authentication bypass.

INPUT:

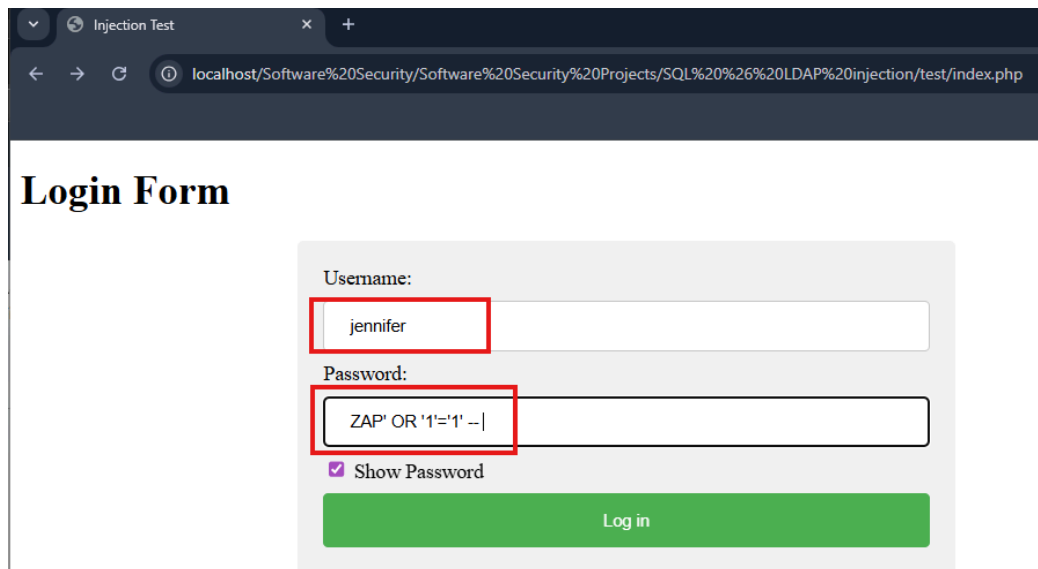
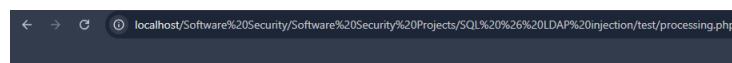


Fig 3.2.1 Login attempt with valid username and password ZAP' OR '1'='1' --

OUTPUT:

The application successfully authenticated the user without validating the actual password.

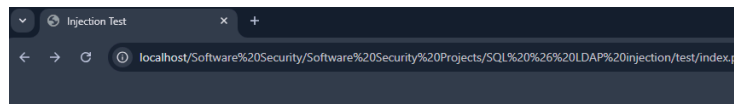


Welcome, jennifer!

Fig 3.2.2 Successful Login with ZAP' OR '1'='1' --

B. Attempt login using a invalid username and password ZAP' OR '1'='1' --

INPUT:



Login Form

Username:

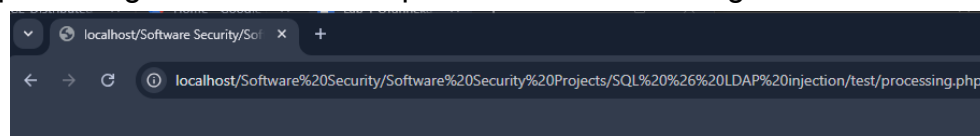
Password:

☒ Show Password

Fig 3.2.3 Login attempt with invalid username and password ZAP' OR '1'='1' --

OUTPUT:

The application granted access despite the username being invalid.



Welcome, Ofunneka!

Fig 3.2.4 Successful Login with invalid username and ZAP' OR '1'='1' --

C Changing user password using a query as input to username field on login form

C.1 - Verify existing password for user 'ismaila' before injection attempt.

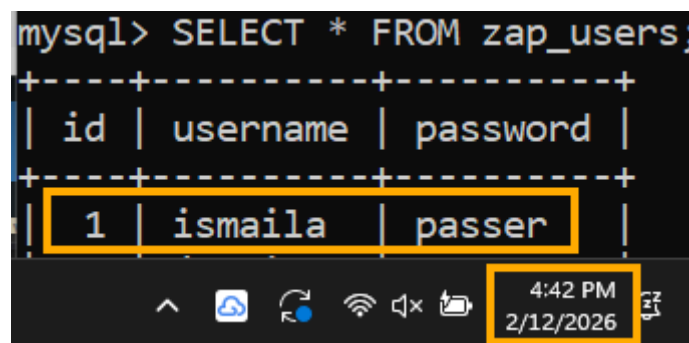


Fig 3.2.5 Verified user password before injection

C.2 - Injection Payload

INPUT

'; UPDATE zap_users SET password='nepasseraplus' WHERE username='ismaila'; --

Username:

`'; UPDATE zap_users SET password='nepasserapulus' WHERE username='ismaila'; --`

Password:

`ZAP' OR '1'='1' --`

☒ Show Password

Log in

4:44 PM 2/12/2026

Fig 3.2.6 SQL update query to modify password used as username input

OUTPUT:

The password for user 'ismaila' was successfully modified in the database

```
mysql> SELECT * FROM zap_users;
```

id	username	password
1	ismaila	nepasserapulus

4:46 PM 2/12/2026

Fig 3.2.7 Password change confirmed after injection.

Step 3 - Verification of Password Change

INPUT: Use the modified password to login

Username:

ismaila

Password:

nepasserapulus

☒ Show Password

Log in

4:50 PM 2/12/2026

Fig 3.2.8 New password used for login attempt

OUTPUT:

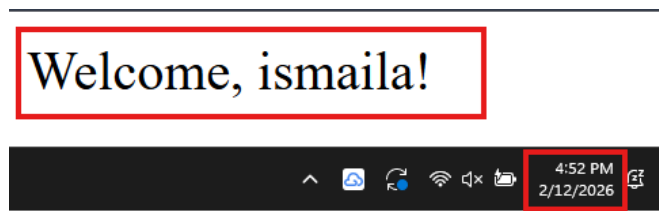


Fig 3.2.9 Successful login with new password

Summary:

The SQL Injection testing conducted confirmed that the login functionality was critically vulnerable. The application allowed authentication bypass, enabling successful login even with invalid credentials. It was also possible to execute arbitrary SQL commands through user input, including modifying a user's password directly within the database. Overall, the vulnerability posed a critical security risk, as it compromised both authentication mechanisms and database integrity.

3.3 Sql Injection Test After Patching/Code Fix

Purpose:

The purpose of this test was to verify that the SQL Injection vulnerability previously identified in the login application had been successfully mitigated after implementing secure coding practices.

The objective was to confirm that authentication bypass and unauthorized database manipulation were no longer possible.

Actions Performed:

A. Fix the vulnerability in processing.php code

The vulnerable SQL query that directly embedded user input into the SQL statement was replaced with a secure implementation using **PDO parameterized queries (prepared statements)**.

This change ensures that user inputs are treated strictly as data and not executable SQL commands.

```
secureprocessing.php X indexS.php
1 <?php
2
3 if ($_SERVER['REQUEST_METHOD'] == 'POST') {
4
5     // Get form input
6     $username = $_POST['username'];
7     $password = $_POST['password'];
8
9     // Database connection parameters
10    $host = "127.0.0.1";
11    $dbname = "zap_test";
12    $dbuser = "janie";
13    $dbpass = "passer";
14
15    try {
16        // Create PDO connection
17        $conn = new PDO("mysql:host=127.0.0.1;port=3309;dbname=$dbname", $dbuser, !
18        #$conn = new PDO("mysql:host=$host;dbname=$dbname", $dbuser, $dbpass);
19        $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
20
21        // Fixed Query (for SQL Injection testing)
22        $query = "SELECT * FROM zap_users
23        WHERE username = :username
24        AND password = :password";
25        $stmt = $conn->prepare($query);
26        $stmt->execute([
27            ':username' => $username,
28            ':password' => $password
29        ]);
30
31        // Fetch results
32        if ($stmt->rowCount() > 0) {
33            echo "Welcome, " . htmlspecialchars($username) . "!";
34        } else {
35            echo "Incorrect username or password.";
36        }
37
38    } catch (PDOException $e) {
39        echo "Database error: " . $e->getMessage();
40    }
41
42 }
43
44 ?>
45
```

Fig 3.3.1 Secure processing.php code

B. Rescan with OWASP ZAP

After applying the code fix, the web application was rescanned using OWASP ZAP to verify remediation.

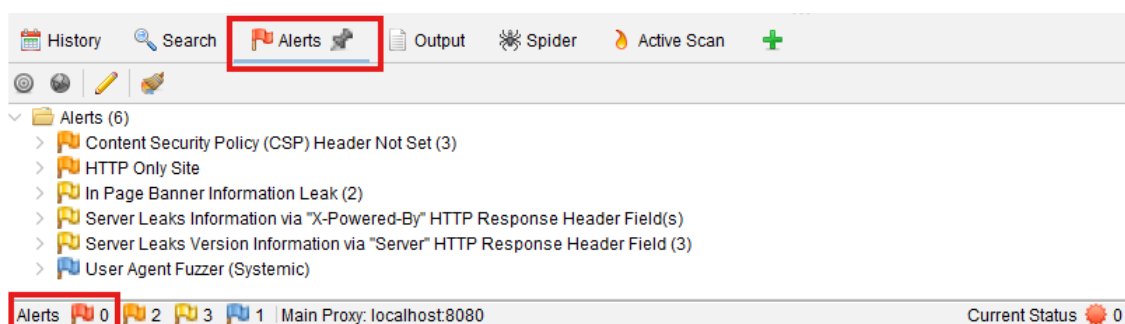


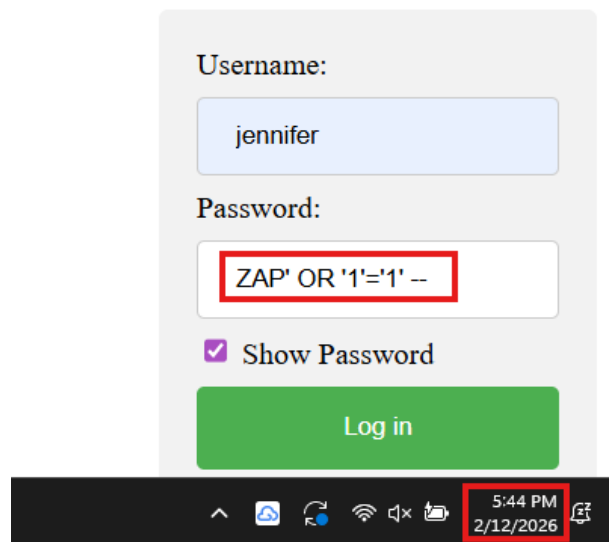
Fig 3.3.2 Scan completed successfully and no SQL Injection vulnerabilities were detected.

C. Login attempt with Valid Username and Password ZAP' OR '1'='1' --

INPUT:

A manual login attempt was performed using the previously successful SQL Injection payload.

Login Form



The screenshot shows a login form with two input fields: 'Username:' and 'Password:'. The 'Username:' field contains the text 'jennifer'. The 'Password:' field contains the SQL injection payload 'ZAP' OR '1'='1' --', which is highlighted with a red rectangular box. Below the password field is a checkbox labeled 'Show Password' with a checked mark. At the bottom of the form is a green button labeled 'Log in'. The form is displayed on a web browser interface, with the system clock in the bottom right corner showing '5:44 PM 2/12/2026'.

Fig 3.3.3 Login attempt with valid username and password ZAP' OR '1'='1' --

OUTPUT:

The login attempt failed, and access was denied.

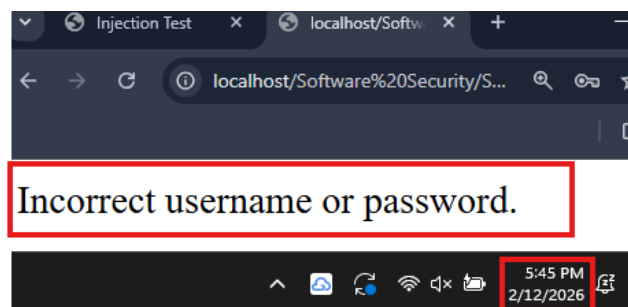


Fig 3.3.4 Failed Login with ZAP' OR '1'='1' --

Result: Authentication bypass was prevented.

D. Password Update attempt through the username input

An attempt was made to modify a user's password by injecting an SQL update statement into the username field.

INPUT:

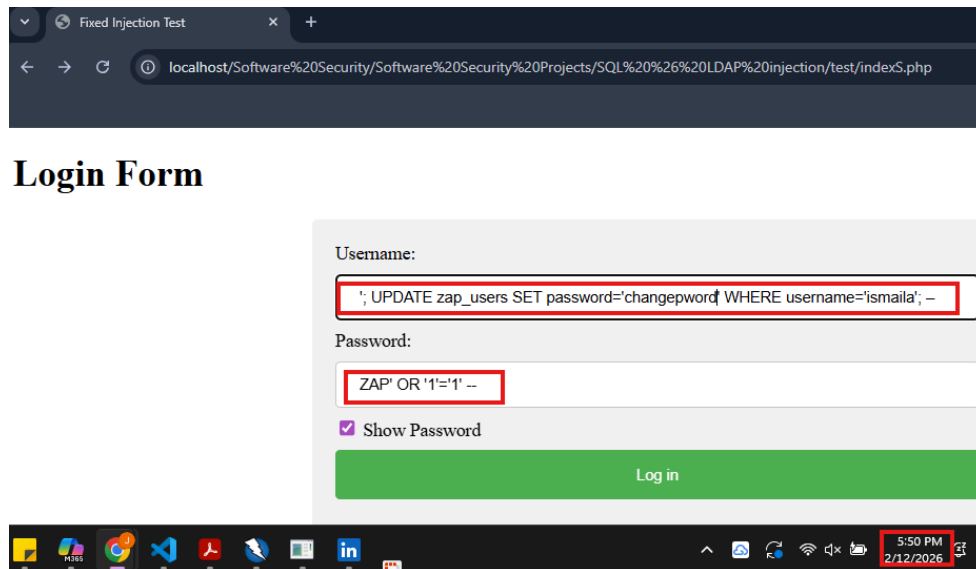


Fig 3.3.5 SQL update query for password modification attempt

OUTPUT:

The application rejected the malicious input, and no database modification occurred.

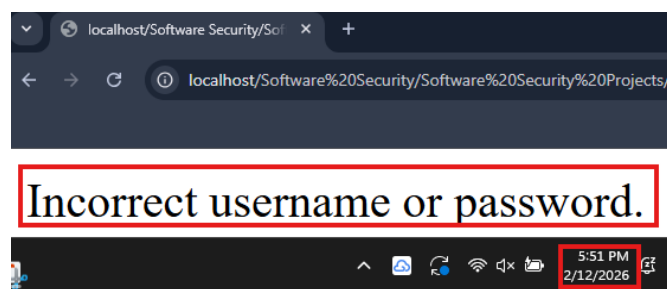


Fig 3.3.6 Failed password modification attempt

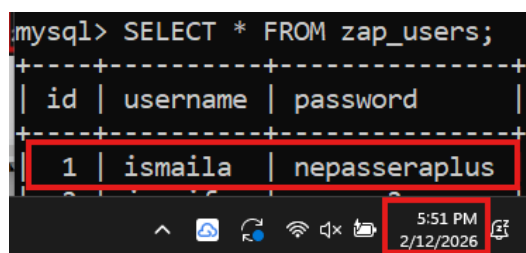


Fig 3.3.7 Password change attempt did not impact database

Summary:

Replacing the vulnerable SQL query with parameterized queries mitigated SQL Injection. Automated scans and manual testing confirmed authentication bypass and unauthorized password modification were prevented, preserving database integrity and secure authentication.

3.4 Conclusion:

SQL injection testing confirmed that the PHP login application allowed authentication bypass and database manipulation. Prepared statements (PDO) successfully mitigated the vulnerabilities, preventing unauthorized access and ensuring secure handling of user input.

4.0 LDAP INJECTIONS

Purpose:

The purpose of this phase is to configure an LDAP-based authentication system, demonstrate LDAP injection vulnerabilities, and implement secure remediation measures to prevent authentication bypass through filter manipulation.

4.1 Environment Configuration

Step 1: Install Required Software

A. Install XAMP Server

Actions Performed:

- Download XAMP from <https://sourceforge.net/projects/xampp/files/>
- Execute the installer, select the components and install
- Launch XAMP and confirm the services are running.

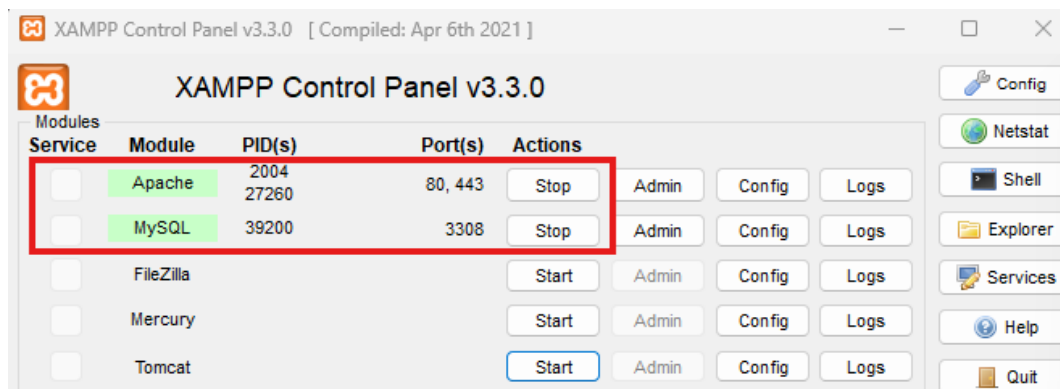


Fig 4.1.1 XAMP Control Panel showing Apache and MySQL running

Step 2: Verify Apache supports LDAP

Purpose:

Actions Performed:

- On the XAMPP control panel, stop the Apache service, click config on the service, and then select **php.ini**.
- Find **ldap** as shown below, and check if it's supported.

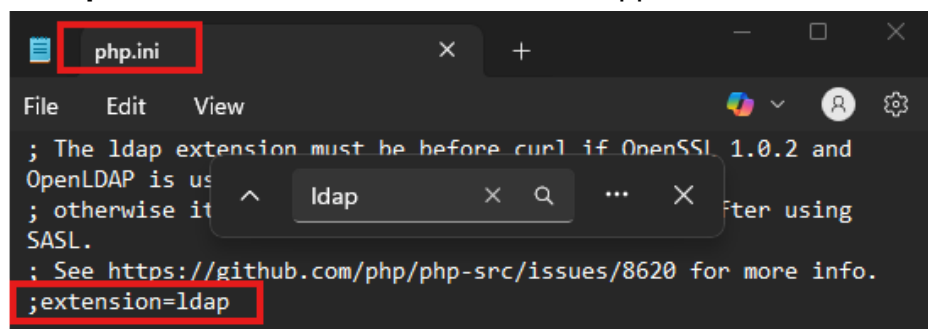


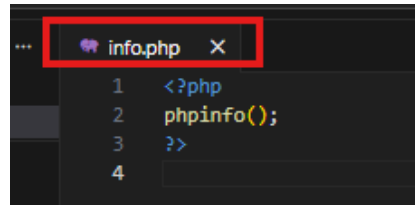
Fig 4.1.2 LDAP extension as found in the Apache php.ini file

→ Uncomment (remove the ;) to enable it.

```
extension=ldap
```

Fig 4.1.3 Uncommented/ LDAP extension enabled

→ On the C:\xampp\htdocs directory, create a file '**info.php**' with content as seen below

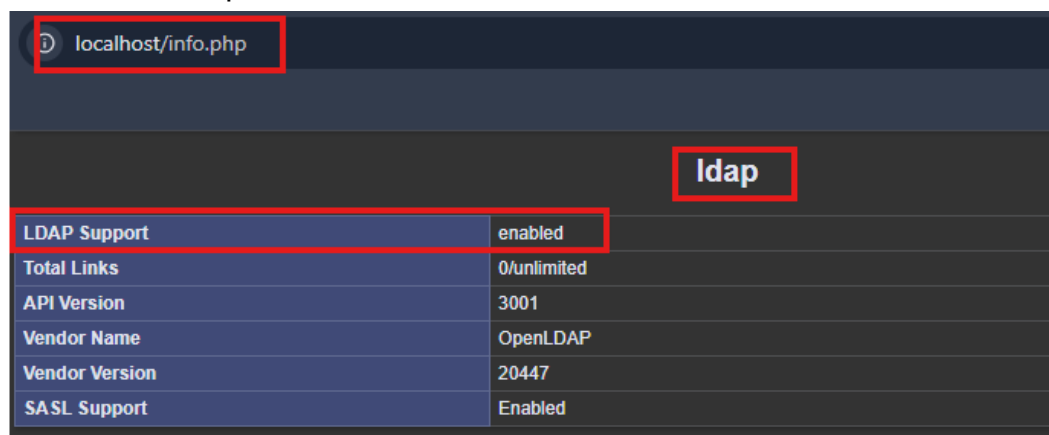


```
1 <?php
2 phpinfo();
3 ?>
4
```

Fig 4.1.4 info.php in the xamp\htdocs directory

→ Start the Apache service on the control panel.

→ On the browser, open the url '<http://localhost/info.php>' and search for the LDAP section. If the section shown below is seen, then the ldap extension is active and ldap functions can be used.



ldap	
LDAP Support	enabled
Total Links	0/unlimited
API Version	3001
Vendor Name	OpenLDAP
Vendor Version	20447
SASL Support	Enabled

Fig 4.1.5 LDAP section as seen on info.php (LDAP support is enabled)

Summary:

The LDAP extension was successfully enabled in Apache by modifying the **php.ini** file and verifying its activation through **info.php**. This confirms that the PHP application can use LDAP functions

Step 3: Create PHP Application in XAMPP Directory

Purpose:

The purpose of this setup is to create a directory inside the XAMP htdocs folder where the vulnerable PHP application will be hosted.

Actions Performed:

A. Create folder named '*LDAP_Injection_Project*' in C:\xampp\htdocs\LDAP_Injection_Project

B. Create *index.php* in the *LDAP_Injection_Project* folder

```
index.php
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <meta http-equiv="content-type" content="text/html; charset=utf-8" />
5   <title>LDAP injections form</title>
6
7   <style>
8     input[type=text],
9     input[type=password] {
10       width: 100%;
11       padding: 12px 20px;
12       margin: 8px 0;
13       display: inline-block;
14       border: 1px solid #ccc;
15       border-radius: 4px;
16       box-sizing: border-box;
17     }
18     input[type=submit] {
19       width: 100%;
20       background-color: #4CAF50;
21       color: white;
22       padding: 14px 20px;
23       margin: 8px 0;
24       border: none;
25       border-radius: 4px;
26       cursor: pointer;
27     }
28     input[type=submit]:hover {
29       background-color: #45a049;
30     }
31     div {
32       width: 50%;
33       border-radius: 5px;
34       background-color: #f2f2f2;
35       padding: 20px;
36     }
37   </style>
38 </head>
39
40 <body>
41   <h1>Login form</h1>
42   <div>
43     <form action="home.php" method="post">
44
45       <label for="username">Username:</label><br>
46       <input type="text" name="username" id="username" required><br>
47
48       <label for="password">Password:</label><br>
49       <input type="password" name="password" id="password" required>
50       <br>
51       <input type="checkbox" onclick="document.getElementById('password').type
52       <br>
53
54       <input type="submit" value="Log in">
55
56     </form>
57   </div>
58 </body>
59 </html>
```

Fig 4.1.6 index.php

C. Create *home.php* in the *LDAP_Injection_Project* folder

```
home.php X
1  <?php
2
3  if ($_SERVER['REQUEST_METHOD'] === 'POST') {
4      // login
5      $ldapServer = 'ldap://10.184.82.55';
6      $ldapPort = 389;
7
8      $ldapConn = ldap_connect($ldapServer, $ldapPort);
9      ldap_set_option($ldapConn, LDAP_OPT_PROTOCOL_VERSION, 3);
10     ldap_set_option($ldapConn, LDAP_OPT_REFERRALS, 0);
11
12     if (!$ldapConn) {
13         die('Unable to connect to LDAP server.');
```

Fig 4.1.7 home.php

Step 4: Setup OpenLDAP Server with Directories

Purpose:

The purpose of this setup is to deploy a centralized directory service using OpenLDAP to manage users, groups, and organizational units (OUs) securely.

Objectives: -

- Install and configure an OpenLDAP server on a different machine on the same network as XAMPP.
- Create a directory structure including OUs, users, and groups.
- Verify the setup using LDAP queries.

Actions Performed:

A. Install OpenLDAP

→ On an **Ubuntu Machine on virtual box**, run the codes:

sudo apt update

sudo apt install slapd ldap-utils -y

OUTPUT

Slapd service installed and running. LDAP utilities available. Run **sudo systemctl status slapd** to confirm status

```
zaphira@zaphira-VirtualBox:~$ sudo systemctl status slapd
● slapd.service - LSB: OpenLDAP standalone server (Lightweight Directory Access Protocol)
   Loaded: loaded (/etc/init.d/slapd; generated)
   Drop-In: /usr/lib/systemd/system/slapd.service.d
            └─slapd-remain-after-exit.conf
   Active: active (running) since Fri 2026-02-13 11:36:07 CET; 3min 5s ago
     Docs: man:systemd-sysv-generator(8)
   Process: 44153 ExecStart=/etc/init.d/slapd start (code=exited, status=0/SUCCESS)
```

Fig 4.1.8 OpenLDAP server installed successfully and running

B. OpenLDAP Configuration

INPUT: Configure slapd with domain and admin password using **sudo dpkg-reconfigure slapd**

```
zaphira@zaphira-VirtualBox:~$ sudo dpkg-reconfigure slapd
Backing up /etc/ldap/slapd.d in /var/backups/slapd-2.6.10+dfsg-0ubuntu0.24.04.1... done.
Moving old database directory to /var/backups:
- directory unknown... done.
Creating initial configuration... done.
Creating LDAP directory... done.
zaphira@zaphira-VirtualBox:~$
```

Fig 4.1.9 LDAP base DN created

OUTPUT: To verify: Run **ldapsearch -x -LLL -H ldap:/// -b dc=ldapinjection,dc=com**

```

zaphira@zaphira-VirtualBox:~$ ldapsearch -x -LLL -H ldap:/// -b dc=ldapinjection,dc=com
dn: dc=ldapinjection,dc=com
objectClass: top
objectClass: dcObject
objectClass: organization
o: ldapinjection
dc: ldapinjection

zaphira@zaphira-VirtualBox:~$

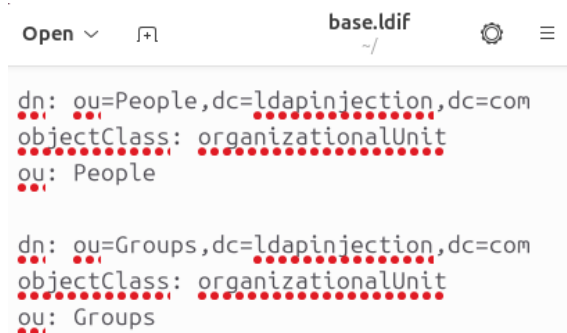
```

Fig 4.1.10 LDAP base DN verified

C. Create Base Directory Structure

INPUT:

→ Create the file **base.ldif** with content below



```

dn: ou=People,dc=ldapinjection,dc=com
objectClass: organizationalUnit
ou: People

dn: ou=Groups,dc=ldapinjection,dc=com
objectClass: organizationalUnit
ou: Groups

```

Fig 4.1.11 base.ldif content

→ Load into LDAP

INPUT:

- Run **ldapadd -x -D cn=admin,dc=ldapinjection,dc=com -W -f base.ldif**
- Enter admin password when prompted

OUTPUT:

Verify using: **ldapsearch -x -LLL -b dc=ldapinjection,dc=com**

```

zaphira@zaphira-VirtualBox:~$ ldapsearch -x -LLL -b dc=ldapinjection,dc=com
dn: dc=ldapinjection,dc=com
objectClass: top
objectClass: dcObject
objectClass: organization
o: ldapinjection
dc: ldapinjection

dn: ou=People,dc=ldapinjection,dc=com
objectClass: organizationalUnit
ou: People

dn: ou=Groups,dc=ldapinjection,dc=com
objectClass: organizationalUnit
ou: Groups

zaphira@zaphira-VirtualBox:~$

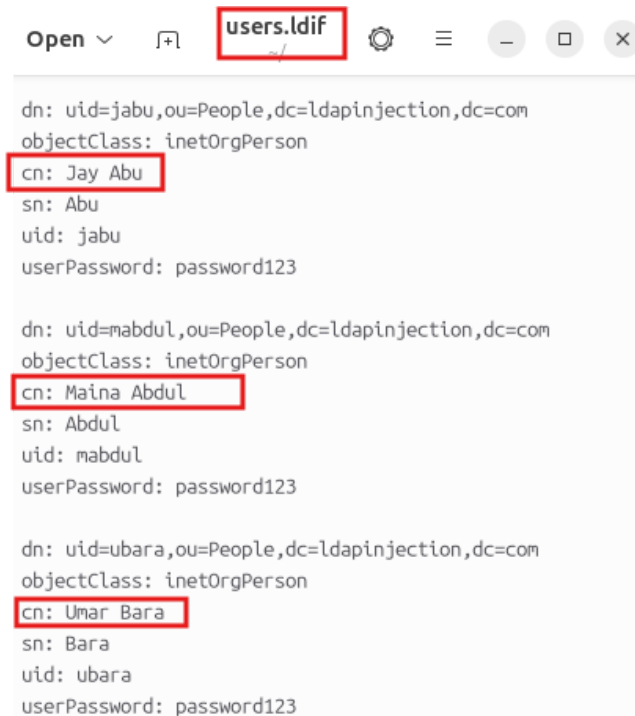
```

Fig 4.1.12 Base directory structure showing people and groups OUs

D. Add Users

INPUT:

→ Create **users.ldif** with content as seen below:



```
Open ▾ [?] users.ldif [Settings] [Menu] [Zoom In] [Zoom Out] [Close]

dn: uid=jabu,ou=People,dc=ldapinjection,dc=com
objectClass: inetOrgPerson
cn: Jay Abu
sn: Abu
uid: jabu
userPassword: password123

dn: uid=mabdul,ou=People,dc=ldapinjection,dc=com
objectClass: inetOrgPerson
cn: Maina Abdul
sn: Abdul
uid: mabdul
userPassword: password123

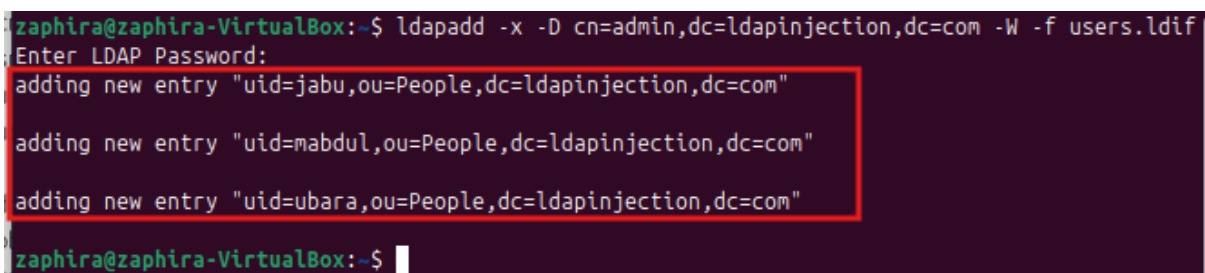
dn: uid=ubara,ou=People,dc=ldapinjection,dc=com
objectClass: inetOrgPerson
cn: Umar Bara
sn: Bara
uid: ubara
userPassword: password123
```

Fig 4.1.13 users.ldif content

→ Load into LDAP

INPUT:

- Run **ldapadd -x -D cn=admin,dc=ldapinjection,dc=com -W -f users.ldif**
- Enter admin password when prompted



```
zaphira@zaphira-VirtualBox:~$ ldapadd -x -D cn=admin,dc=ldapinjection,dc=com -W -f users.ldif
Enter LDAP Password:
adding new entry "uid=jabu,ou=People,dc=ldapinjection,dc=com"
adding new entry "uid=mabdul,ou=People,dc=ldapinjection,dc=com"
adding new entry "uid=ubara,ou=People,dc=ldapinjection,dc=com"
zaphira@zaphira-VirtualBox:~$
```

Fig 4.1.14 Users created in **People** OU

OUTPUT:

Verify using: **ldapsearch -x -LLL -b ou=People,dc=ldapinjection,dc=com**

```

zaphira@zaphira-VirtualBox:~$ ldapsearch -x -LLL -b ou=People,dc=ldapinjection,dc=com
dn: ou=People,dc=ldapinjection,dc=com
objectClass: organizationalUnit
ou: People

dn: uid=jabu,ou=People,dc=ldapinjection,dc=com
objectClass: inetOrgPerson
cn: Jay Abu
sn: Abu
uid: jabu

dn: uid=ubara,ou=People,dc=ldapinjection,dc=com
objectClass: inetOrgPerson
cn: Umar Bara
sn: Bara
uid: ubara

dn: uid=mabdul,ou=People,dc=ldapinjection,dc=com
objectClass: inetOrgPerson
cn: Maina Abdul
sn: Abdul
uid: mabdul

```

Fig 4.1.15 Returned users with their attributes.

OUTPUT 2:

Verify search by uid using: **ldapsearch -x -D "cn=admin,dc=ldapinjection,dc=com" -W -b "dc=ldapinjection,dc=com" "(uid=ubara)"**

```

zaphira@zaphira-VirtualBox:~$ ldapsearch -x -D "cn=admin,dc=ldapinjection,dc=com" -W -b "dc=ldapinjection,dc=com" "(uid=ubara)"
Enter LDAP Password:
# extended LDIF
#
# LDAPv3
# base <dc=ldapinjection,dc=com> with scope subtree
# filter: (uid=ubara)
# requesting: ALL
#
# ubara, People, ldapinjection.com
dn: uid=ubara,ou=People,dc=ldapinjection,dc=com
objectClass: inetOrgPerson
cn: Umar Bara
sn: Bara
uid: ubara
userPassword:: cGFzc3dvcmQxMjM=
# search result
search: 2
result: 0 Success

# numResponses: 2
# numEntries: 1
zaphira@zaphira-VirtualBox:~$

```

Fig 4.1.16 Returned attributes for uid 'ubara'

Summary:

The OpenLDAP server was successfully installed and configured on the Ubuntu virtual machine. The base DN (dc=ldapinjection,dc=com) was created, organizational units were added, and user accounts were successfully loaded into the directory. LDAP queries confirmed that the directory structure and user entries were properly configured.

Step 5: Confirm Connection between XAMPP Application & LDAP Directory

Purpose:

The purpose of this step is to verify successful network communication between the XAMPP-hosted PHP application and the OpenLDAP server running on the Ubuntu virtual machine.

Confirming this connectivity ensures that the web application can send LDAP queries and receive responses from the directory service before proceeding with authentication activities.

Actions Performed:

A. Find IP for VM hosting the LDAP server

INPUT:

→ On an **Ubuntu Machine on virtual box**, run the command:

Ip a

OUTPUT

```
link/ether 08:00:27:9b:9f:ce brd ff:ff:ff:ff:ff:ff
inet 10.184.82.55/24 brd 10.184.82.255 scope global dynamic noprefixroute
```

Fig 4.1.17 Retrieved IP of LDAP Server VM

B. Test Connectivity from Host

INPUT:

→ On local machine, run **ping 10.184.82.55**

OUTPUT:

Successful ping/connection to VM server

```
C:\Users\Starlix>ping 10.184.82.55

Pinging 10.184.82.55 with 32 bytes of data:
Reply from 10.184.82.55: bytes=32 time=3ms TTL=64
Reply from 10.184.82.55: bytes=32 time=1ms TTL=64
Reply from 10.184.82.55: bytes=32 time=1ms TTL=64
Reply from 10.184.82.55: bytes=32 time=2ms TTL=64

Ping statistics for 10.184.82.55:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 3ms, Average = 1ms

C:\Users\Starlix>
```

Fig 4.1.18 Successful ping connection to VM IP

INPUT:

→ To confirm connection on ldap port, run **telnet 10.184.82.55 389**

OUTPUT:

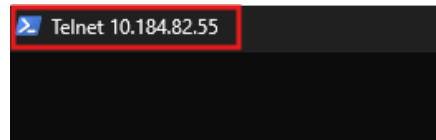


Fig 4.1.19 Successful telnet connection via ldap port 389

Summary:

Successful ping and telnet tests confirmed that the XAMPP-hosted PHP application could communicate with the OpenLDAP server over port 389. This ensured that authentication testing could proceed without network connectivity issues.

4.2 LDAP Authentication

Purpose:

The purpose of this phase is to validate LDAP-based authentication from the PHP application and confirm that legitimate users stored in the directory can successfully authenticate.

Actions Performed:

Step 1: Verify Authentication

Confirm that valid LDAP users can authenticate through the PHP login form.

INPUT:

- Access form with url http://localhost/LDAP_Injection_Project/home.php
- Fill login form with username and password saved on LDAP directory
- Login

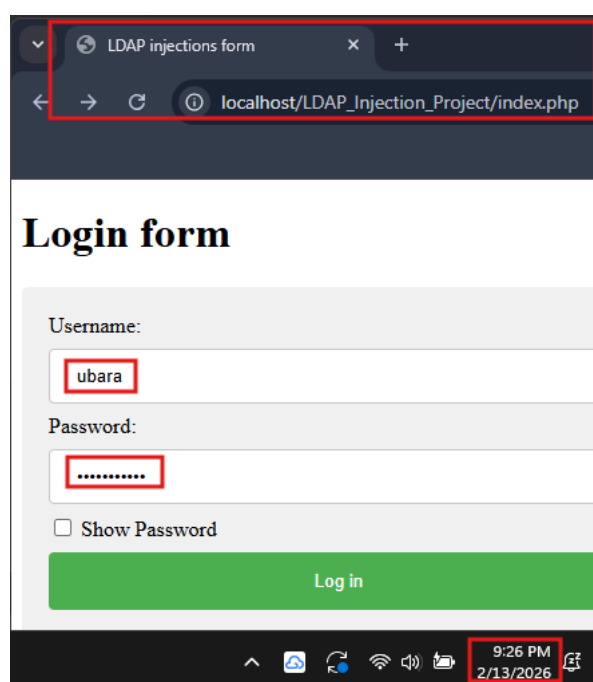


Fig 4.2.1 Login form

OUTPUT:

Login successful.

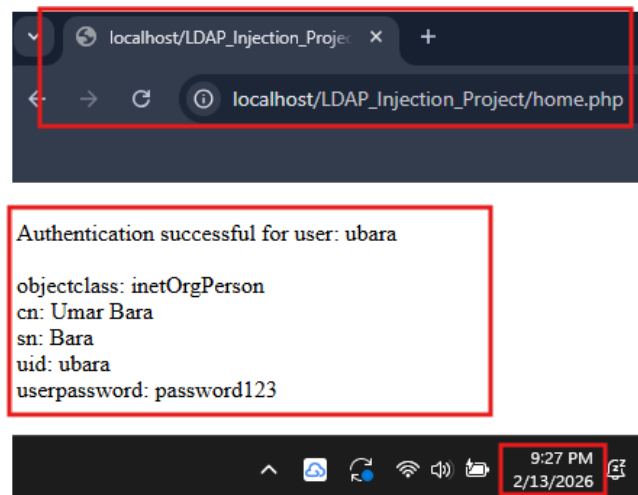


Fig 4.2.2 Successful Login/Authentication

Summary

Authentication using valid LDAP user credentials was successful.. This confirms that the PHP application properly communicates with the OpenLDAP server and that the authentication mechanism functions as expected.

4.3 Perform LDAP Injection Bypass

Purpose:

The purpose of this phase is to demonstrate authentication bypass via LDAP injection.

Actions Performed:

INPUT 1: Injection Input Using Wildcard Characters (*)

Objective: To test whether LDAP query allows broad matching of directory entries.

- Username is *
- Password is *
- Target filter is (&(uid=*)(userPassword=*))

Login form

A screenshot of a login form. It has two input fields: 'Username:' and 'Password:'. Both fields contain an asterisk (*) and are highlighted with red boxes. Below the password field is a checkbox labeled 'Show Password' which is checked. At the bottom is a green 'Log in' button. The system tray at the bottom right shows the time '10:11 PM' and date '2/13/2026'.

Fig 4.3.1 Injection Input Using * for both username and password

OUTPUT 1:

Authentication bypass succeeded

Authentication successful for user: *

objectclass: inetOrgPerson
cn: Jay Abu
sn: Abu
uid: jabu
userpassword: password123



Fig 4.3.2 Successful Authentication for * Injection

Malformed Logical Injection Attempt

Objective: Login attempt using injected logical operators and unbalanced parentheses to manipulate the LDAP filter structure.

INPUT 2:

Username is **ubara)(!(&(**

Password is **any))**

Target filter is **(&(uid = ubara)(!(& (|) (userPassword =any))))**

Login form

A screenshot of a login form on a mobile device. The form has two input fields: 'Username:' and 'Password:'. The 'Username' field contains the text 'ubara)(!(&(and the 'Password' field contains 'any))'. Both inputs are highlighted with red boxes. Below the password field is a checkbox labeled 'Show Password' which is checked. At the bottom of the form is a green button labeled 'Log in'. The mobile status bar at the bottom shows the time as 11:06 PM and the date as 2/13/2026, which are highlighted with a red box.

Fig 4.3.3 Login attempt using injected logical operators and parenthesis manipulation

OUTPUT 2:

Authentication was granted even with manipulated logical operators, indicating improper input sanitization and vulnerable filter construction.

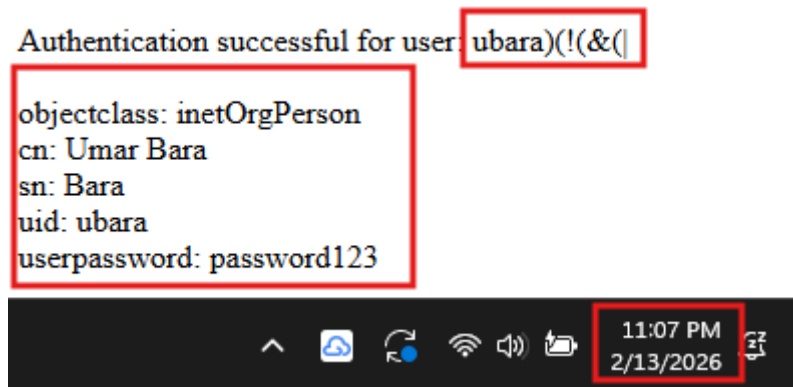


Fig 4.3.4 Successful Authentication despite filter manipulation

INPUT 3:

Username is ubara)(!(&(

Password is passera))

Target filter is (&(uid=ubara)(!(&(!))(userPassword=passera)))

Login form

Username:

ubara)(!(&(

Password:

passera))

☒ Show Password

Log in

7:48 PM
2/14/2026

Fig 4.3.5 Injection using additional closing parentheses with wrong password

OUTPUT 3:

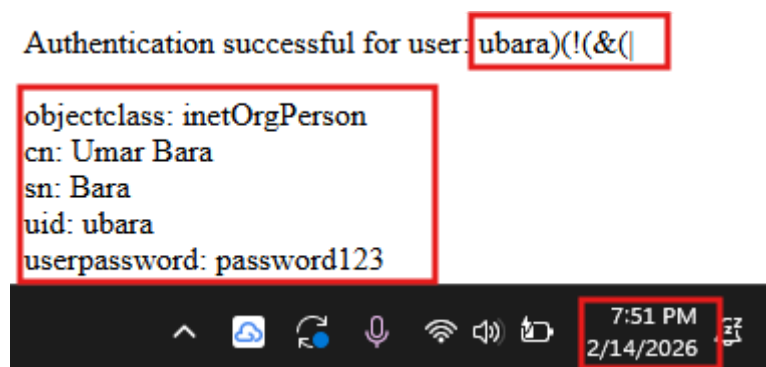


Fig 4.3.6 Successful login despite manipulated filter structure

INPUT 4: Logical Operator Injection Payloads

Objective: To inject logical operators to manipulate filter conditions.

Username is *)(&
Password is *)(&
Target filter is (&(uid=*)(&(userPassword=*)(&))

A screenshot of a web application's login page. The 'Username:' field contains the payload '*)(&' and the 'Password:' field also contains '*)(&'. Both fields are highlighted with red boxes. Below the password field is a checked checkbox labeled 'Show Password'. A green 'Log in' button is at the bottom. The mobile status bar at the bottom shows the time '11:27 PM' and date '2/13/2026', which are also highlighted with red boxes.

Fig 4.3.7 Injection using empty logical conditions *)(& as username and password

OUTPUT 4:

Authentication successful for user: *)(&
objectclass: inetOrgPerson
cn: Jay Abu
sn: Abu
uid: jabu
userpassword: password123

A screenshot of the mobile status bar at the bottom of the screen, showing the time '11:28 PM' and date '2/13/2026', which are highlighted with red boxes.

Fig 4.3.8 Successful authentication despite malformed logic conditions

Summary

The results demonstrate that the application is vulnerable to LDAP injection due to unsanitized user input and direct concatenation into LDAP filters. All tested payloads resulted in authentication bypass, confirming the absence of proper input validation and secure filter construction mechanisms.

4.4 Vulnerability Fix

Purpose:

To eliminate LDAP Injection vulnerabilities by ensuring all user-supplied input is sanitized before being incorporated into LDAP search filters.

Issue to be fixed:

The application constructs LDAP filters by directly concatenating user input without sanitizing the input values. This made it possible to inject LDAP operators (*, (,), &, |, !) and manipulate filter logic, leading to authentication bypass.

Corrective Action Implemented

- Implement input sanitization using the PHP `ldap_escape()` function with the `LDAP_ESCAPE_FILTER` flag.

Action Performed:

In `home.php` file, replace the search filter

```
$ldapSearchFilter = "(&(uid=$username)(userPassword=$password))";
```

with

```
39 $ldapSearchFilter = "(&(uid=" . ldap_escape($username, "", LDAP_ESCAPE_FILTER) . ") .  
40 "(userPassword=" . ldap_escape($password, "", LDAP_ESCAPE_FILTER) . "))";  
41
```

4.5 Verification and Re-Testing

Objective:

Confirm that LDAP injection vulnerabilities have been successfully mitigated.

Action Performed:

- Re-execute all previously successful injection inputs
 - Wildcat (*) injection
 - Parenthesis manipulation payloads
 - Logical operator injection payloads
- Observe application response

INPUT

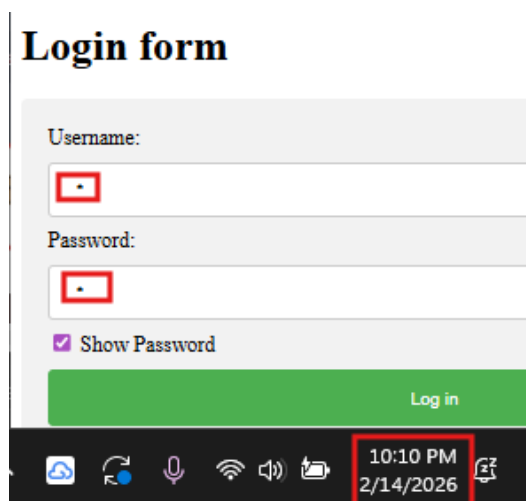


Fig 4.5.1

OUTPUT

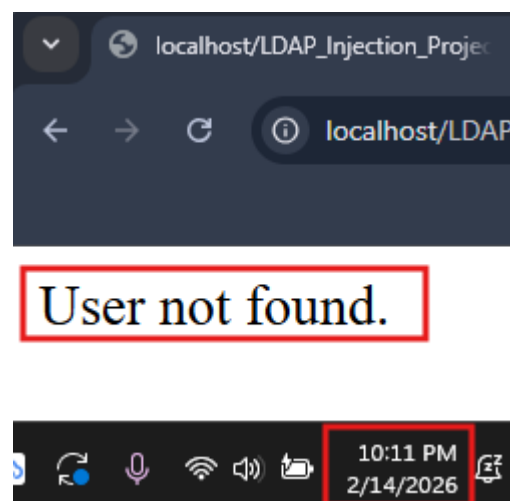
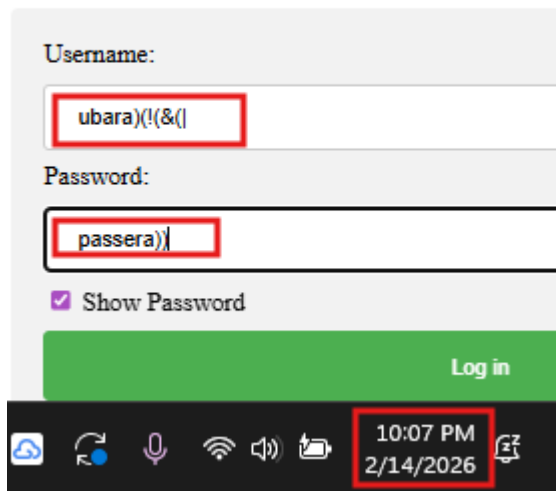


Fig 4.5.2 Authentication failed.

Login form



A screenshot of a web application's login form. The form has two input fields: 'Username:' and 'Password:'. The 'Username' field contains the text 'ubara)(!(&(|' and the 'Password' field contains 'passera))'. Below the password field is a checkbox labeled 'Show Password' which is checked. A green 'Log in' button is at the bottom of the form. The system tray at the bottom of the screen shows the time as 10:07 PM on 2/14/2026.

Fig 4.5.3

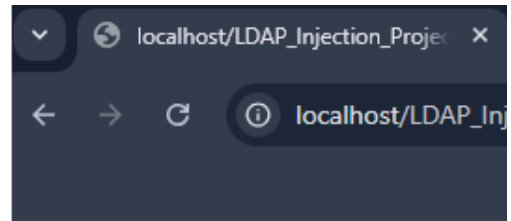
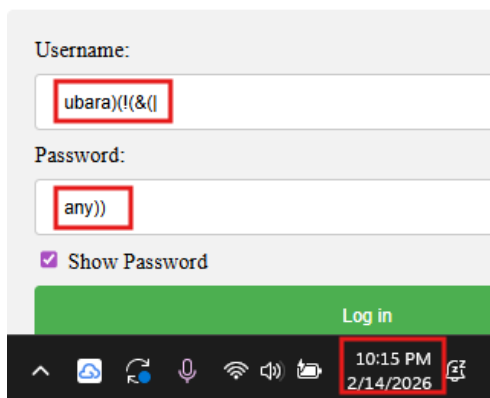


Fig 4.5.4 Authentication also failed.

Login form



A screenshot of a web application's login form. The 'Username' field contains 'ubara)(!(&(|' and the 'Password' field contains 'any))'. The 'Show Password' checkbox is checked. A green 'Log in' button is at the bottom. The system tray shows the time as 10:15 PM on 2/14/2026.

Fig 4.5.5

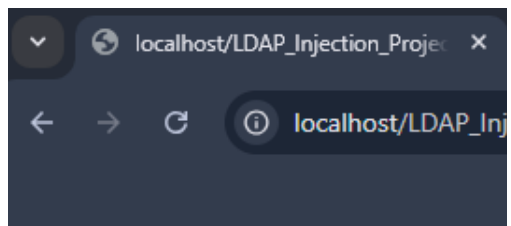
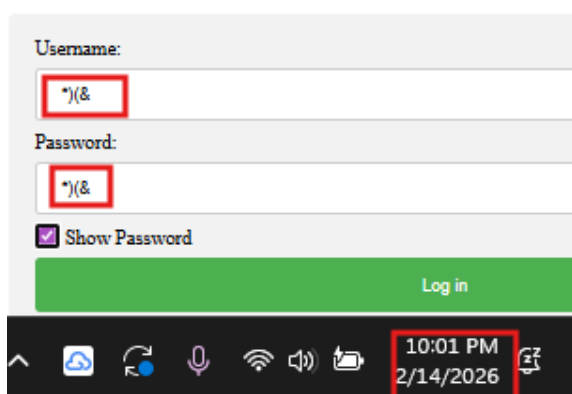


Fig 4.5.6 Failed Authentication

Login form



A screenshot of a web application's login form. The 'Username' field contains '*)(&' and the 'Password' field contains '*)(&'. The 'Show Password' checkbox is checked. A green 'Log in' button is at the bottom. The system tray shows the time as 10:01 PM on 2/14/2026.

Fig 4.5.7

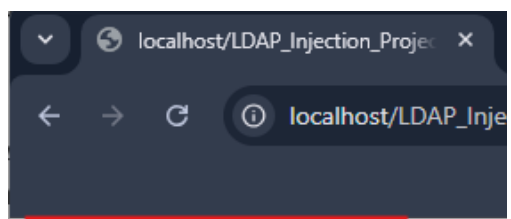


Fig 4.5.8 Failed Authentication

All previously successful injection attempts failed after implementation of `ldap_escape()`. No authentication bypass was possible.

Summary:

The implementation of `ldap_escape()` successfully mitigates LDAP Injection vulnerabilities within the authentication module. The login mechanism now processes only properly sanitized input, preventing unauthorized access via filter manipulation.

4.6 Conclusion:

LDAP injection testing showed that unsanitized input could manipulate LDAP filters to bypass authentication. Applying `ldap_escape()` effectively prevented all injection attempts, restoring proper authentication and securing filter logic.

FINAL CONCLUSION

This project successfully demonstrated both SQL Injection and LDAP Injection vulnerabilities within deliberately insecure PHP authentication systems. Each component - the PHP application, MySQL database, and OpenLDAP server - played a clearly defined role, making the system modular, testable, and secure after remediation.

The SQL injection phase showed that direct concatenation of user input into SQL queries allows authentication bypass and unauthorized database manipulation. This vulnerability was eliminated through the implementation of PDO prepared statements, ensuring strict separation between data and SQL logic.

The LDAP injection phase demonstrated how unsanitized input can manipulate LDAP filter logic to bypass authentication using wildcards, logical operators, and parenthesis manipulation. This vulnerability was successfully mitigated using PHP's `ldap_escape()` function with the `LDAP_ESCAPE_FILTER` flag.

SQL injection vulnerabilities were identified through automated scanning using OWASP ZAP and manually verified via authentication bypass testing. LDAP injection vulnerabilities were demonstrated entirely through manual input testing using various injection payloads. Both vulnerabilities were effectively remediated.

The implementation covered:

- Development environment setup (WAMP/XAMPP, Ubuntu OpenLDAP VM).
- Database and directory configuration.
- Deployment of intentionally vulnerable PHP authentication forms.
- Manual testing of SQL and LDAP injection payloads to demonstrate authentication bypass.
- Application of secure coding practices including PDO parameterized queries and LDAP input escaping.
- Verification and retesting to confirm vulnerability mitigation.

The resulting system supports:

- Secure authentication with MySQL and LDAP backends.
- Prevention of authentication bypass.
- Robust input validation and sanitization to ensure the integrity of database queries and LDAP filters.

This project highlights the importance of secure coding practices, input validation, parameterized queries, and proper escaping mechanisms in preventing injection-based attacks.

Finally, this operational document provides a step-by-step guide, allowing the system to be reliably reproduced and serving as a reference for both vulnerability demonstration and secure remediation.

APPENDIX

Appendix A – SQL Injection Payloads Used

- ZAP' OR '1'='1' –
- '; UPDATE zap_users SET password='nepasseraplus' WHERE username='ismaila'; --

Appendix B – LDAP Injection Payloads Used

Injection Type	Username	Password
Wildcard	*	*
Logical/Parenthesis	ubara)(!((&(	any))
Logical/Parenthesis	ubara)(!((&(	passera))
Logical Operator	*) (&	*) (&

Appendix C – Secure Coding Techniques Implemented

SQL Protection:

- PDO Prepared Statements
- Parameter Binding

LDAP Protection:

- ldap_escape() with LDAP_ESCAPE_FILTER

Appendix D – Table of Findings

Section	Vulnerability	Test Method	Outcome Before Fix	Fix Applied	Outcome After Fix
SQL Injection	Authentication bypass & arbitrary SQL execution	OWASP ZAP scan + manual login attempts	Login succeeded with invalid credentials; user passwords could be modified via injection	PDO prepared statements (parameterized queries)	Login only succeeds with valid credentials; SQL commands from input are ignored
LDAP Injection	Authentication bypass via LDAP filter manipulation	Manual login with payloads: *, logical operators, parenthesis manipulation	All payloads bypassed authentication	PHP ldap_escape() with LDAP_ESCAPE_FILTER	All previous injection attempts failed; authentication works correctly and securely